ARIA: Adaptive Recurrent Integrative Architecture

A Unified Predictive Framework for Perception, Memory, and Reasoning

Harshit Saxena

Abstract

Contemporary deep learning architectures for general intelligence tasks treat perception, memory, and reasoning as fundamentally separate computational problems. Perception is handled by encoders, memory is approximated by fixed-length context windows or retrieval-augmented databases, and reasoning emerges implicitly through successive forward passes or is offloaded to chain-of-thought prompting. We argue that this modular decomposition is both architecturally inefficient and fundamentally misaligned with the computational principles of biological cognition, where these three faculties arise as manifestations of a single underlying process: *hierarchical predictive state estimation over multiple timescales*. We introduce **ARIA** (Adaptive Recurrent Integrative Architecture), a neural architecture grounded in predictive coding theory, hierarchical temporal abstraction, and complementary memory systems. ARIA features a hierarchy of Predictive State Columns (PSCs) operating at geometrically increasing timescales, precision-weighted bidirectional message passing (top-down predictions and bottom-up prediction errors), a tripartite memory substrate comprising episodic, working, and semantic stores with a biologically-inspired consolidation cycle, and adaptive recurrent depth that allocates computation proportionally to problem difficulty. We present the complete mathematical framework, derive the training objective from variational free energy minimization, analyze theoretical computational properties including adaptive sparsity and temporal abstraction guarantees, propose a concrete four-phase implementation roadmap with detailed experimental protocols, and discuss open problems and failure modes. While ARIA remains a theoretical proposal, we argue that its unifying principle—that all cognition is prediction error minimization at multiple timescales with memory consolidation—offers a principled north star for next-generation architectures that transcend the limitations of current monolithic attention-based models.

Keywords: predictive coding, hierarchical temporal memory, free energy principle, adaptive computation, continual learning, working memory, neural architecture design

1 Introduction

1.1 The Fragmentation Problem

The modern deep learning landscape for general-purpose AI is dominated by the Transformer architecture [Vaswani et al., 2017] and its descendants. Despite remarkable empirical success across language modeling [Brown et al., 2020, Touvron et al., 2023], vision [Dosovitskiy et al., 2021], multimodal understanding [Alayrac et al., 2022], and increasingly agentic reasoning [Yao et al., 2023], the Transformer embodies a set of architectural assumptions that impose fundamental structural limitations.

The canonical Transformer pipeline can be expressed as:

$$y = \text{Decoder}\big(\text{Attention}(\text{Encoder}(x), \text{ Context Window})\big). \tag{1}$$

In this formulation, three cognitive functions are handled by distinct mechanisms:

(i) **Perception** is the encoder's job: mapping raw input tokens to distributed representations.

(ii) **Memory** is approximated by the context window: a fixed-length buffer of previous token representations that the attention mechanism can query.

(iii) **Reasoning** emerges implicitly as a byproduct of successive attention layers, or is explicitly elicited through prompting strategies such as chain-of-thought [Wei et al., 2022].

This separation creates well-documented failure modes. The fixed context window imposes a hard boundary on temporal integration [Press et al., 2022]. The absence of persistent, mutable memory means the system cannot learn from interaction without full retraining, and even retrieval-augmented approaches [Lewis et al., 2020] treat memory as an external database rather than an integral component of computation. Reasoning remains shallow and difficult to scale, as evidenced by systematic failures on compositional and multi-step problems [Dziri et al., 2024], and the depth of reasoning is architecturally fixed at the number of layers regardless of problem difficulty.

These are not merely engineering problems amenable to incremental solutions. They reflect a deeper architectural misalignment: the treatment of perception, memory, and reasoning as separable modules, when the evidence from neuroscience, cognitive science, and computational theory suggests they are inseparable aspects of a single process.

1.2 The Biological Precedent

The mammalian neocortex does not implement perception, memory, and reasoning as separate subsystems. Consider the act of reading a sentence. Recognizing letterforms requires memory retrieval (matching visual patterns to learned grapheme representations). Understanding word meanings requires reasoning (disambiguating based on context, integrating prior knowledge, and generating inferences). And the inferences drawn reshape perception itself—expectations about upcoming words influence how ambiguous visual input is processed [Kuperberg and Jaeger, 2016].

This unification reflects the deep computational architecture of the cortex, which, according to a growing body of theoretical and empirical work [Rao and Ballard, 1999, Friston, 2005, 2010, Clark, 2013, Keller and Mrsic-Flogel, 2018], implements a hierarchical predictive coding scheme in which:

- Higher cortical areas generate *top-down predictions* of expected activity in lower areas.
- Lower areas compute *prediction errors*—the difference between actual and predicted activity—and propagate these errors upward.
- *Perception* is the process of explaining away prediction errors by updating internal models.
- *Memory* is embedded in synaptic weights (semantic memory), hippocampal traces (episodic memory), and sustained prefrontal activity (working memory).
- *Reasoning* is extended inference—the same predictive machinery running without external input, using working memory to maintain and manipulate variable bindings.

Furthermore, the cortex operates over multiple timescales simultaneously [Murray et al., 2014, Hasson et al., 2015, Chaudhuri et al., 2015]. This multi-timescale hierarchy is architecturally imposed through intrinsic neural time constants that vary systematically across the cortical hierarchy.

1.3 The Proposal

We propose **ARIA** (Adaptive Recurrent Integrative Architecture), a neural architecture that takes the unification of perception, memory, and reasoning as its foundational design principle. ARIA is built on three core axioms:

Axiom 1: Prediction is the universal currency. The system actively predicts its inputs at every level of a processing hierarchy, and only the prediction errors—the surprises—drive further computation. This implements the free energy minimization principle [Friston, 2005] as an architectural prior.

Axiom 2: Memory is a state, not a database. Memory is integral to the system's computational state, manifesting in three complementary forms: fast episodic traces, capacity-limited working memory registers, and slow parametric (semantic) knowledge—with a consolidation process that migrates information between these stores.

Axiom 3: Reasoning is variable-depth recursion. The depth of computation is adaptive, not fixed. Simple inputs that are well-predicted require minimal processing; complex or novel inputs require deep processing. Reasoning is what happens when this recurrent loop runs without external input, using working memory to maintain intermediate results.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 presents the full architecture. Section 4 derives the mathematical framework. Section 5 analyzes theoretical properties. Section 6 proposes the implementation roadmap. Section 7 discusses limitations and open problems. Section 8 concludes.

2 Related Work

2.1 Predictive Coding and the Free Energy Principle

The theoretical foundation of ARIA is the predictive coding framework [Rao and Ballard, 1999, Friston, 2005] and the free energy principle [Friston, 2010]. Key aspects incorporated into ARIA include: bidirectional message passing between hierarchical levels [Rao and Ballard, 1999]; precision weighting of prediction errors, allowing the system to learn how much to trust different information sources [Feldman and Friston, 2010]; and active inference, in which the system can act on the environment to fulfill its predictions [Friston et al., 2016].

Prior neural implementations of predictive coding include PredNet [Lotter et al., 2017], deep predictive coding networks [Wen et al., 2018], predictive coding approximations to backpropagation [Millidge et al., 2022], and biologically plausible credit assignment surveys [Salvatori et al., 2024]. These implementations are generally limited to perceptual processing in a single modality and have not attempted the full unification with memory and reasoning that ARIA proposes.

2.2 Hierarchical Temporal Processing

Neuroscientific evidence establishes a systematic gradient of intrinsic timescales across the cortical hierarchy [Murray et al., 2014, Hasson et al., 2015, Chaudhuri et al., 2015]. In machine learning, hierarchical temporal processing has been explored through Clockwork RNNs [Koutník et al., 2014], Hierarchical Multiscale RNNs [Chung et al., 2017], Temporal Convolutional Networks [van den Oord et al., 2016, Bai et al., 2018], and State Space Models [Gu et al., 2022, Gu and Dao, 2023]. ARIA differs by coupling multi-timescale dynamics with predictive coding and using architectural (fixed) clock rates to provide a strong inductive bias for temporal abstraction.

2.3 Memory-Augmented Neural Networks

The limitations of fixed-context-window models have motivated extensive work on augmented memory: Neural Turing Machines [Graves et al., 2014], Differentiable Neural Computers [Graves et al., 2016], Memory Networks [Weston et al., 2015, Sukhbaatar et al., 2015], Retrieval-Augmented Generation [Lewis et al., 2020, Borgeaud et al., 2022], and Memorizing Transformers [Wu et al., 2022]. The neuroscience-inspired complementary learning systems (CLS) theory [McClelland et al., 1995, Kumaran et al., 2016] has been implemented via elastic weight consolidation [Kirkpatrick

et al., 2017], experience replay [Mnih et al., 2015, Schaul et al., 2016], and continual learning methods [Zenke et al., 2017, Lopez-Paz and Ranzato, 2017]. ARIA integrates these ideas into a unified memory substrate with a consolidation cycle.

2.4 Adaptive Computation

Adaptive Computation Time [Graves, 2016], Universal Transformers [Dehghani et al., 2019], PonderNet [Banino et al., 2021], Mixture of Experts [Shazeer et al., 2017, Fedus et al., 2022], and early exit strategies [Teerapittayanon et al., 2016, Schwartz et al., 2020] all explore variable computational cost. ARIA's adaptive computation emerges naturally from predictive coding dynamics rather than requiring a separate halting mechanism.

2.5 Recurrent and State-Space Architectures

Recent SSM architectures—S4 [Gu et al., 2022], Mamba [Gu and Dao, 2023], RWKV [Peng et al., 2023], RetNet [Sun et al., 2023], and Griffin [De et al., 2024]—demonstrate competitive performance with efficient recurrence. ARIA leverages SSM-like dynamics within each level but embeds them within a broader predictive coding framework with cross-level communication, memory integration, and variable-depth recursion.

3 Architecture

3.1 Overview

ARIA consists of three major components:

(1) A **Predictive Hierarchy** of L levels (baseline $L = 4$), each implementing a Predictive State Column (PSC) at a different timescale.

(2) A **Memory Substrate** comprising three complementary stores (episodic, working, semantic) accessible to all levels.

(3) An **Adaptive Computation Controller** that determines when processing has converged and output can be emitted.

Top-down connections carry predictions; bottom-up connections carry prediction errors; lateral connections implement state transitions. The full system architecture is illustrated in Figure 1.

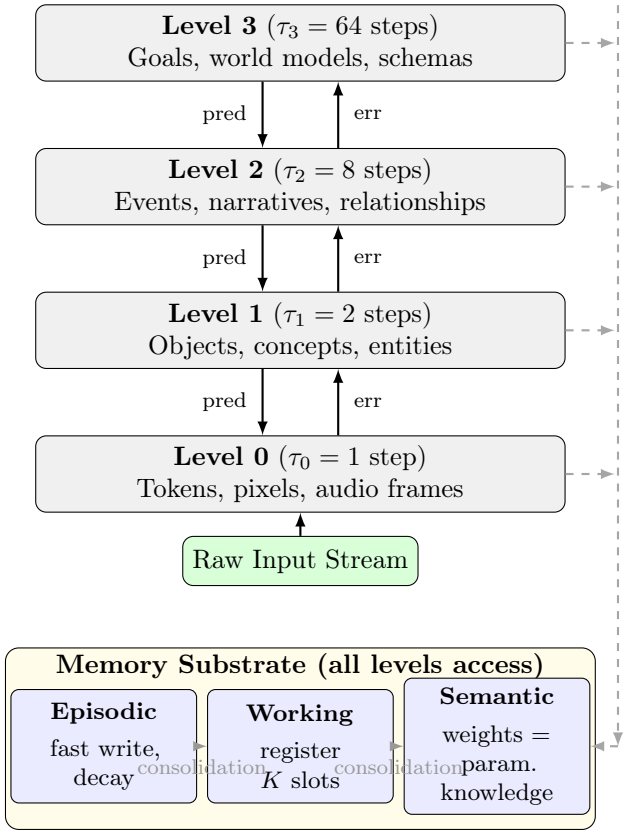

Figure 1: ARIA architecture overview. Solid arrows between levels indicate top-down predictions (left, pointing downward) and bottom-up prediction errors (right, pointing upward). The dashed vertical bus connects all levels to the Memory Substrate. Within the substrate, dashed arrows show the consolidation flow from episodic through working to semantic memory.

3.2 The Predictive State Column (PSC)

The fundamental computational unit of ARIA is the Predictive State Column (PSC), inspired by the cortical column and the predictive processing unit proposed in theoretical neuroscience [Bastos et al., 2012].

Definition 3.1 (Predictive State Column)**.** *A PSC at level $l \in \{0, 1, \ldots, L-1\}$ is a tuple* $\mathrm{PSC}_l = (d_l,\, \tau_l,\, f_l,\, g_l,\, q_l,\, \pi_l)$ *where:*

- $d_l \in \mathbb{N}$ *is the state dimensionality.*
- $\tau_l \in \mathbb{N}$ *is the clock period (the PSC updates only at timesteps t where $t \bmod \tau_l = 0$).*
- $f_l : \mathbb{R}^{D^l_{\text{in}}} \to \mathbb{R}^{d_l}$ *is the state transition function.*
- $g_l : \mathbb{R}^{d_l} \to \mathbb{R}^{d_{l-1}}$ *is the downward prediction function.*
- $q_l : \mathbb{R}^{d_l} \to \mathbb{R}^{d_q}$ *is the memory query generator.*
- $\pi_l : \mathbb{R}^{d_l} \to \mathbb{R}^3_+$ *is the precision estimator, outputting non-negative precision weights.*

Clock periods follow a geometric progression $\tau_l = \tau_0 \cdot r^l$ with $\tau_0 = 1$ and ratio $r > 1$. We use $\tau \in \{1, 2, 8, 64\}$ for the four-level baseline.

The state update rule for level l at timestep t is:

$$s_l(t{+}1) = \begin{cases} s_l(t) + \alpha_l \cdot f_l\big(s_l(t),\ \pi_l^{\uparrow} \odot \varepsilon_{l-1}(t),\ \pi_l^{\downarrow} \odot g_{l+1}(s_{l+1}(t)),\ \pi_l^{m} \odot m_l(t),\ w(t)\big) & \text{if } t \bmod \tau_l = 0, \\ s_l(t) & \text{otherwise,} \end{cases} \tag{2}$$

where:

- $s_l(t) \in \mathbb{R}^{d_l}$ is the state vector at level l and time t.
- $\alpha_l \in (0, 1]$ is a learnable step size.
- $\varepsilon_{l-1}(t) = s_{l-1}(t) - g_l(s_l(t))$ is the prediction error from level $l-1$.
- $g_{l+1}(s_{l+1}(t))$ is the top-down prediction from level $l+1$.
- $m_l(t) = \mathrm{Read}(M_{\mathrm{ep}}, q_l(s_l(t)))$ is the memory read result.
- $w(t) = [w_1(t); \ldots; w_K(t)]$ is the concatenated working memory.
- $(\pi_l^{\uparrow}, \pi_l^{\downarrow}, \pi_l^{m}) = \pi_l(s_l(t))$ are the precision weights.
- $\odot$ denotes element-wise multiplication.

Boundary conditions: Level 0 receives raw input prediction error $\varepsilon_{-1}(t) \triangleq x(t) - g_0(s_0(t))$, and Level $L-1$ uses $g_L \equiv 0$.

3.3 Implementation of PSC Sub-Networks

State Transition Function f_l. We implement f_l as a gated recurrent block:

$$h_l = \mathrm{Concat}\big(s_l(t),\ \pi_l^{\uparrow} \odot \varepsilon_{l-1}(t),\ \pi_l^{\downarrow} \odot \hat{s}_l^{\downarrow}(t),\ \pi_l^{m} \odot m_l(t),\ w(t)\big), \tag{3}$$

$$z_l = \sigma(W_z^l h_l + b_z^l) \quad \text{(update gate)}, \tag{4}$$

$$r_l = \sigma(W_r^l h_l + b_r^l) \quad \text{(reset gate)}, \tag{5}$$

$$\tilde{s}_l = \phi\big(W_s^l [r_l \odot s_l(t);\ \varepsilon_{l-1}(t);\ \hat{s}_l^{\downarrow}(t);\ m_l(t);\ w(t)] + b_s^l\big), \tag{6}$$

$$f_l(\cdot) = (1 - z_l) \odot s_l(t) + z_l \odot \tilde{s}_l - s_l(t), \tag{7}$$

where ϕ is SiLU [Elfwing et al., 2018] and $\hat{s}_l^{\downarrow}(t) = g_{l+1}(s_{l+1}(t))$ is the top-down prediction. For levels with larger state dimensions, a local self-attention block (window size w) is additionally applied within the state transition:

$$\tilde{s}_l \leftarrow \tilde{s}_l + \mathrm{LocalAttention}_w(\tilde{s}_l). \tag{8}$$

Downward Prediction Function g_l. g_l is a two-layer MLP:

$$g_l(s_l) = W_g^l \cdot \phi(W_{g_1}^l s_l + b_{g_1}^l) + b_g^l. \tag{9}$$

Memory Query Generator q_l.

$$q_l(s_l) = W_q^l s_l + b_q^l. \tag{10}$$

Precision Estimator π_l.

$$\pi_l(s_l) = \mathrm{Softplus}(W_\pi^l s_l + b_\pi^l) \in \mathbb{R}_+^{3 \times d_l}. \tag{11}$$

The three output groups correspond to $\pi_l^{\uparrow}$, $\pi_l^{\downarrow}$, and π_l^{m}.

Table 1: Timescale hierarchy in the four-level ARIA baseline.

Level	Clock Period τ_l	Effective Receptive Field	Cognitive Scope
0	1	1 step	Individual tokens / frames
1	2	∼2–4 steps	Phrases, local patterns
2	8	∼16–64 steps	Paragraphs, events
3	64	∼128–4096 steps	Documents, goals

3.4 Multi-Timescale Dynamics

Proposition 3.2 (Temporal Abstraction)**.** *A level l PSC with clock period τ_l and state dimensionality d_l, receiving prediction errors from level $l-1$ with clock period $\tau_{l-1} < \tau_l$, is forced to represent information that varies on timescales $\geq \tau_l$, since it cannot track faster variations.*

Proof sketch. By the Nyquist–Shannon sampling theorem, a signal sampled at rate $1/\tau_l$ can faithfully represent frequency components up to $1/(2\tau_l)$. Since the level updates only every τ_l steps, information varying faster than τ_l is aliased or averaged, and the level's state must capture slower-varying structure to minimize prediction errors across its update intervals. □

3.5 The Memory Substrate

3.5.1 Episodic Memory (M_{ep})

Episodic memory provides fast, one-shot storage of specific experiences, inspired by modern Hopfield networks [Ramsauer et al., 2021] and differentiable memory systems [Graves et al., 2016]. The buffer maintains N_{ep} entries, each consisting of a key vector $k_i \in \mathbb{R}^{d_k}$, a value vector $v_i \in \mathbb{R}^{d_v}$, an importance weight $\omega_i \in \mathbb{R}_+$, and a timestamp $t_i \in \mathbb{N}$.

Write Operation.

$$k_{\text{new}} = W_k \cdot \text{Concat}(s_0(t), s_1(t), \ldots, s_{L-1}(t)), \tag{12}$$

$$v_{\text{new}} = W_v \cdot \text{Concat}(s_0(t), \ldots, s_{L-1}(t), w(t)), \tag{13}$$

$$\omega_{\text{new}} = \sigma\big(u^\top \cdot \text{Concat}(\varepsilon_0(t), \ldots, \varepsilon_{L-1}(t))\big). \tag{14}$$

Read Operation. Given a query $q \in \mathbb{R}^{d_k}$:

$$\alpha_i = \frac{\exp(\beta \cdot \text{sim}(q, k_i))}{\sum_j \exp(\beta \cdot \text{sim}(q, k_j))}, \tag{15}$$

$$\text{Read}(M_{\text{ep}}, q) = \sum_i \alpha_i v_i, \tag{16}$$

where $\text{sim}(q, k) = q^\top W_{\text{sim}} k$ (bilinear similarity followed by layer normalization) and β is a temperature parameter.

Forgetting Policy.

$$\text{Retain}(i) = \omega_i \cdot \exp(-\lambda \cdot (t_{\text{current}} - t_i)) + \gamma \cdot \text{RecentAccessCount}(i). \tag{17}$$

Algorithm 1 Consolidation

```
procedure CONSOLIDATE(M_ep, Θ, n_replay, λ_EWC)
    E ← PrioritizedSample(M_ep, n_replay)
    for each episode e ∈ E do
        s ← ForwardPass(e.context, Θ)
        L_replay ← PredictionLoss(s, e.targets)
        L_EWC ← Σ_i F_i · (θ_i − θ*_i)^2
        L_total ← L_replay + λ_EWC · L_EWC
        Θ ← Θ − η_consolidate · ∇_Θ L_total
    end for
    for each episode e ∈ E do
        e.ω ← e.ω · decay_factor
    end for
end procedure
```

3.5.2 Working Memory ($\mathcal{W}$)

Working memory consists of $K = 7$ slots (informed by Miller's 7 ± 2 capacity limit [Miller, 1956]):

$$\mathcal{W} = \{w_1, w_2, \ldots, w_K\}, \quad w_k \in \mathbb{R}^{d_w}. \tag{18}$$

Only levels $l \geq l_{\text{min}}$ (we set $l_{\text{min}} = 2$) can write to working memory, reflecting top-down executive control. A writing level l generates:

$$(\text{slot}, \text{content}, \text{strength}) = \text{WriteHead}_l(s_l(t)), \tag{19}$$

$$w_k(t+1) = (1 - \text{strength} \cdot a_k) \cdot w_k(t) + \text{strength} \cdot a_k \cdot \text{content}, \tag{20}$$

where a_k is a soft attention weight over slots.

3.5.3 Semantic Memory (Θ)

Semantic memory is the network's parameters Θ—the slowest-changing store, representing consolidated general knowledge. Unlike standard deep learning, Θ can be updated continuously through the consolidation process (Section 3.6), enabling lifelong learning.

3.6 The Consolidation Cycle

Consolidation migrates information from episodic to semantic memory, implementing the complementary learning systems principle [McClelland et al., 1995]. The procedure is described in Algorithm 1.

Key design choices include: prioritized replay of high-importance episodes [Schaul et al., 2016]; Elastic Weight Consolidation [Kirkpatrick et al., 2017] to prevent catastrophic forgetting; interleaved replay across time periods; and a small consolidation learning rate $\eta_{\text{consolidate}} \ll \eta_{\text{train}}$.

3.7 Adaptive Computation and Convergence

Definition 3.3 (Convergence)**.** *The system has converged at time t if*

$$\Delta(t) = \sum_{l=0}^{L-1} \lambda_l \|s_l(t) - s_l(t - \tau_l)\|_2 < \delta_{\text{converge}}, \tag{21}$$

$$C(t) = \text{Confidence}(s_{L-1}(t), \mathcal{W}(t)) > \delta_{\text{confidence}}. \tag{22}$$

Algorithm 2 Adaptive Inference

```
procedure INFER(input_sequence, max_iterations)
    Initialize s_l ← s_l^(0) for all l
    for each input x(t) in input_sequence do
        for iter = 1 to max_iterations do
            for l = 0 to L − 1 do
                if t mod τ_l = 0 then
                    s_l ← PSC_l.step(s_l, ε_{l−1}, pred_{l+1}, memory, W)
                end if
            end for
            if Δ(t) < δ_converge and C(t) > δ_confidence then
                break
            end if
        end for
        if output requested then
            output ← OutputHead(s_0, ..., s_{L−1}, W)
        end if
    end for
end procedure
```

The adaptive inference procedure is described in Algorithm 2. **Internal reasoning mode** is identical except $x(t) = \emptyset$: the recurrent dynamics continue driven by top-down predictions, memory retrievals, and working memory manipulation, constituting reasoning as extended recurrence.

4 Mathematical Framework

4.1 Generative Model

ARIA implicitly defines a hierarchical generative model. At each level l:

$$p(s_{l-1} \mid s_l) = \mathcal{N}(s_{l-1};\ g_l(s_l),\ \Pi_l^{-1}), \tag{23}$$

where $\Pi_l = \mathrm{diag}(\pi_l)$ is the precision matrix. The prior over the highest level is $p(s_{L-1}) = \mathcal{N}(s_{L-1}; 0, \Pi_{L-1}^{-1})$, and the full generative model factors as:

$$p(x, s_0, \ldots, s_{L-1}) = p(x \mid s_0) \prod_{l=1}^{L-1} p(s_{l-1} \mid s_l) \cdot p(s_{L-1}). \tag{24}$$

4.2 Variational Free Energy

Given observations x, the system maintains an approximate posterior $q(s_0, \ldots, s_{L-1})$ parameterized by the current state vectors. The variational free energy is:

$$F = D_{\mathrm{KL}}[q(s) \| p(s)] - \mathbb{E}_q[\log p(x \mid s_0)]. \tag{25}$$

Under the Laplace approximation, this simplifies to:

$$F \approx \underbrace{\sum_{l=0}^{L-1} \tfrac{1}{2}\, \varepsilon_l^\top \Pi_l \varepsilon_l}_{\mathcal{L}_{\text{pred}}} + \underbrace{\tfrac{1}{2} \sum_{l=0}^{L-1} \log |\Pi_l^{-1}|}_{\mathcal{L}_{\text{complexity}}}, \tag{26}$$

where $\varepsilon_l = s_l - g_{l+1}(s_{l+1})$ (and $\varepsilon_0 = x - g_0(s_0)$).

4.3 State Dynamics as Gradient Descent on Free Energy

A key result from predictive coding theory [Friston, 2005, Buckley et al., 2017] is that state update rules can be derived as gradient descent on the free energy:

$$\dot{s}_l = -\frac{\partial F}{\partial s_l} = \Pi_l \varepsilon_l - \Pi_{l-1} \left(\frac{\partial g_l}{\partial s_l} \right)^\top \varepsilon_{l-1}. \tag{27}$$

ARIA parameterizes the state dynamics with neural networks (the transition function f_l) rather than using the exact gradient, providing greater expressiveness while maintaining the same qualitative structure.

4.4 Training Objective

The total training loss combines the variational free energy with task-specific losses:

$$\mathcal{L}_{\text{total}} = \underbrace{\sum_{l=0}^{L-1} \lambda_l \|\varepsilon_l\|^2_{\Pi_l}}_{\mathcal{L}_{\text{pred}}} + \underbrace{\beta \cdot D_{\text{KL}}[q(s)\|p(s)]}_{\mathcal{L}_{\text{complexity}}} + \underbrace{\gamma \cdot \mathcal{L}_{\text{task}}}_{\text{task loss}} + \underbrace{\delta \cdot \mathcal{L}_{\text{compute}}}_{\text{compute cost}}, \tag{28}$$

where the compute cost follows PonderNet [Banino et al., 2021]:

$$\mathcal{L}_{\text{compute}} = D_{\text{KL}}[p_{\text{halt}}(n)\|\text{Geom}(\lambda_p)]. \tag{29}$$

4.5 Training with Truncated Backpropagation Through Time

We use level-specific truncation windows proportional to clock periods:

$$T_l^{\text{trunc}} = \min(T_{\max},\ c \cdot \tau_l), \tag{30}$$

where $c = 4$ in our baseline. Fast levels have short truncation windows; slow levels have longer windows, enabling learning from extended temporal dependencies. Synthetic gradients [Jaderberg et al., 2017] can optionally be used at truncation boundaries to reduce bias.

4.6 Precision Learning

Proposition 4.1 (Optimal Precision)**.** *Under the generative model of Section 4, the optimal precision for level l's bottom-up prediction error is proportional to the inverse variance of the prediction error:*

$$\pi_l^{*\uparrow} = \frac{1}{\text{Var}[\varepsilon_{l-1}]}. \tag{31}$$

Proof. This follows directly from maximum likelihood estimation of the precision parameter in a Gaussian model. □

5 Theoretical Analysis

5.1 Computational Efficiency via Predictive Sparsity

Theorem 5.1 (Adaptive Sparsity)**.** *Let $\varepsilon_l(t)$ be the prediction error at level l and time t, and let η_l be the threshold below which errors are not propagated. In a well-trained ARIA system with empirical predictability ρ_l at level l (fraction of timesteps where $\|\varepsilon_l(t)\| < \eta_l$), the expected computational cost per timestep is:*

$$\text{Cost}(t) = \sum_{l=0}^{L-1} \frac{1}{\tau_l} \cdot \left(c^l_{\text{base}} + (1 - \rho_l) \cdot c^l_{\text{error}} \right). \tag{32}$$

Corollary 5.2. *For natural language with Zipfian frequency distributions, the expected computational savings relative to uniform processing are approximately:*

$$\textit{Savings} \approx 1 - \sum_{l=0}^{L-1} \frac{1-\rho_l}{\tau_l}. \tag{33}$$

For typical values ($\rho_0 \approx 0.3, \rho_1 \approx 0.5, \rho_2 \approx 0.7, \rho_3 \approx 0.9$) *and* $\tau = \{1, 2, 8, 64\}$*, this yields approximately* 72% *savings over uniform full-depth processing.*

5.2 Memory Capacity and Forgetting

Proposition 5.3 (Episodic Memory Effective Capacity)**.** *The effective number of retrievable entries (entries with retention score above threshold* η*) at time* T *is:*

$$N_{\text{eff}}(T) = \min\left(N_{\text{ep}},\ \frac{\bar{\omega}}{\lambda} \cdot \log \frac{\bar{\omega}}{\eta}\right), \tag{34}$$

where λ *is the decay rate and* $\bar{\omega}$ *is the mean importance weight.*

Proposition 5.4 (Consolidation Convergence)**.** *Under mild regularity conditions on the replay distribution and learning rate schedule, the consolidation process converges:*

$$\mathbb{E}[\|\Theta(t+1) - \Theta(t)\|^2] \leq \frac{\eta^2_{\text{consolidate}} \cdot \sigma^2_{\text{grad}}}{t} + \mathcal{O}(\eta^2_{\text{consolidate}}). \tag{35}$$

5.3 Reasoning Depth and Computational Completeness

Theorem 5.5 (Computational Universality)**.** *An ARIA system with* $K \geq 2$ *working memory slots, unbounded iteration count, and transition functions with sufficient capacity is Turing-complete.*

Proof sketch. The working memory slots simulate a finite tape segment, the recurrent state dynamics simulate state transitions, and the iteration mechanism provides unbounded computation time. This follows from the Turing completeness of RNNs with unbounded computation [Siegelmann and Sontag, 1995], extended to our specific architecture. □

Proposition 5.6 (Depth-Difficulty Scaling)**.** *For a family of problems with difficulty measure* D*, a well-trained ARIA system will use approximately* $\mathcal{O}(D)$ *iterations to solve problems, assuming the transition functions have learned an effective inference strategy and the convergence criterion is appropriately calibrated.*

5.4 Relationship to Attention

Proposition 5.7 (Precision as Structured Attention)**.** *The precision-weighted integration in ARIA's state update can be viewed as a form of attention where: the "queries" are implicit in the current state via* $\pi_l(s_l(t))$*; the "keys" and "values" are the prediction errors* ε_{l-1}*, top-down predictions* $g_{l+1}(s_{l+1})$*, and memory contents* m_l*; and the "attention weights" are the precision values* $\pi_l^{\uparrow}, \pi_l^{\downarrow}, \pi_l^{m}$*.*

Unlike Transformer attention, precision attention operates over information types rather than sequence positions, and is derived from a principled uncertainty estimation framework.

6 Implementation Roadmap and Experimental Design

6.1 Phase 1: Proof of Concept (Language Only)

Goal. Demonstrate that ARIA achieves competitive performance on standard language modeling benchmarks, with qualitative evidence of adaptive computation and hierarchical prediction.

Table 2: Phase 1 architecture specification.

Component	Specification
Levels	$L = 4$
State dimensions	$d_0 = d_1 = 512$, $d_2 = d_3 = 768$
Clock periods	$\tau = \{1, 2, 8, 64\}$
Transition function	Gated recurrence + 2-head local attention (window = 32)
Prediction function	2-layer MLP, SiLU activation
Precision estimator	1-layer linear + Softplus
Episodic memory	$N_{\text{ep}} = 65\,536$ entries, $d_k = 128$, $d_v = 512$
Working memory	$K = 7$ slots, $d_w = 256$
Total parameters	∼350M
Max iterations / token	16
Convergence thresholds	$\delta_{\text{converge}} = 0.01$, $\delta_{\text{confidence}} = 0.9$

Architecture Specification.

Training Protocol. Data: RedPajama / The Pile (∼100B tokens). Optimizer: AdamW [Loshchilov and Hutter, 2019] with cosine schedule, peak LR 3×10^{-4}, weight decay 0.1, gradient clip at 1.0. BPTT truncation windows $T_l^{\text{trunc}} = \{16, 16, 32, 64\}$. Loss weights: $\lambda_l = \{1.0, 0.5, 0.25, 0.125\}$, $\beta = 0.01$, $\gamma = 1.0$, $\delta = 0.001$. Training curriculum starts at max_iterations = 2, gradually increasing to 16.

Ablation Studies.

(1) No hierarchy ($L = 1$).

(2) No precision weighting (fixed $\pi = 1$).

(3) No multi-timescale (all $\tau_l = 1$).

(4) No episodic memory.

(5) No working memory.

(6) Fixed iterations (no adaptive computation).

Success Criteria. Perplexity within 10% of same-size Transformer; clear adaptive computation signal; hierarchical levels capturing different temporal scales; each ablation showing measurable degradation.

6.2 Phase 2: Continuous Learning

Goal. Demonstrate that ARIA can absorb new knowledge from interaction without catastrophic forgetting.

Key Experiments. (1) *Fact Absorption*: novel facts presented conversationally, tested at varying recall intervals and after buffer overflow. (2) *Error Correction*: corrections to existing knowledge, with measurement of persistence and interference. (3) *Continual Learning Benchmark*: forward/backward transfer on established benchmarks, compared to EWC-only and replay-only baselines. (4) *Long-Term Knowledge Accumulation*: extended training stream, measuring retention curve and consolidation efficiency.

Success Criteria. Facts retained across sessions; catastrophic forgetting reduced by $\geq 50\%$ versus naive fine-tuning; successful episodic-to-semantic migration.

6.3 Phase 3: Multimodal Extension

Goal. Demonstrate that ARIA's predictive coding framework naturally extends to multimodal processing via cross-modal prediction.

Modality-specific encoders are added at Level 0, while Levels 1–3 are shared across modalities. Cross-modal binding occurs through shared higher-level states, cross-modal prediction, and prediction error as a binding signal.

Key Experiments. Vision-language understanding (VQA, captioning, visual grounding); cross-modal prediction quality; multimodal reasoning over diagrams and text.

6.4 Phase 4: Embodiment and Active Inference

Goal. Extend ARIA to an embodied agent learning through active inference.

The optimal action minimizes expected future free energy:

$$a^* = \arg\min_a \mathbb{E}_{q(s'|a)}[F(s', x')]. \tag{36}$$

The action head is trained via imagined rollouts:

$$\mathcal{L}_{\text{action}} = \sum_{h=1}^{H} \mathbb{E}[F(t+h) \mid a(t), \ldots, a(t+h-1)]. \tag{37}$$

Key Experiments. Exploration efficiency in simulated embodied environments; instruction following; cross-environment transfer.

7 Discussion

7.1 Strengths and Potential Advantages

1. **Unified Framework.** Perception, memory, and reasoning arise from a single computational principle, avoiding engineering complexity and potential inconsistencies of modular systems.
2. **Adaptive Efficiency.** Prediction error propagation provides natural adaptive computation without a separate halting mechanism.
3. **Continuous Learning.** Architecturally integrated consolidation addresses the inability of current systems to learn from deployment-time interactions.
4. **Uncertainty Awareness.** Precision weighting provides native uncertainty estimation, potentially reducing hallucination.
5. **Biological Plausibility.** Architectural principles that recur across diverse neural systems provide validated inductive biases.

7.2 Limitations and Open Problems

Training Stability. The combination of recurrent dynamics, hierarchical message passing, and memory operations may be difficult to optimize. Precision weights could collapse ($\pi \to 0$) or explode ($\pi \to \infty$). Mitigations: careful initialization, gradient clipping, curriculum learning, spectral normalization.

Credit Assignment. Truncated BPTT introduces bias; memory operations involve near-discrete decisions. Mitigations: synthetic gradients [Jaderberg et al., 2017], straight-through estimators [Bengio et al., 2013], two-phase training.

Memory Management at Scale. Buffer size limits temporal recall range; retrieval cost scales with buffer size; no clearly optimal forgetting policy. Mitigations: hierarchical indexing, approximate nearest-neighbor retrieval (e.g., FAISS; Johnson et al., 2019), learned forgetting policies via reinforcement learning.

Sufficiency of Prediction Error. The brain also has reward signals, emotional valence, and neuromodulatory systems that are not captured by prediction error alone. Mitigations: explicit reward signals, curiosity bonuses, learned neuromodulatory gating.

Scaling Properties. ARIA introduces multiple interacting scaling dimensions (hierarchy depth, state dimensions, memory capacity, iteration count) whose composition into scaling laws is currently unknown. Mitigation: systematic empirical scaling studies.

Engineering Complexity and Parallelizability. Sequential recurrent dynamics and variable-length adaptive computation complicate batched inference. Mitigations: scan-based SSM implementations, padding-based batching, efficient key-value stores.

Evaluation Methodology. Standard benchmarks may not capture ARIA's intended advantages. New metrics are needed for adaptive computation efficiency, hierarchical representation quality, continuous learning, and uncertainty calibration.

7.3 Comparison with Concurrent Approaches

Table 3: Comparison of ARIA with major architecture families.

Feature	Transformer	Mamba/SSM	ARIA
Core mechanism	Self-attention	Selective state space	Predictive state column
Memory	Context window	Recurrent state	Episodic + Working + Semantic
Temporal processing	Position encoding	Continuous dynamics	Multi-timescale hierarchy
Computation / input	Fixed	Fixed	Adaptive
Continual learning	No	No	Yes (consolidation)
Uncertainty estimation	External calibration	No	Native (precision)
Reasoning mechanism	Chain-of-thought	Not explicit	Adaptive recurrence + WM
Scaling behavior	Well understood	Emerging	Unknown
Training efficiency	High (parallel)	High (scan)	Lower (sequential)
Engineering complexity	Low	Medium	High
Biological plausibility	Low	Medium	High

7.4 Ethical Considerations

Continuous learning risks. A system that learns from interaction could absorb biased, toxic, or factually incorrect information. Safeguards include content filtering on memory writes, adversarial robustness testing, and selective memory deletion mechanisms.

Computational equity. Adaptive computation that allocates more processing to surprising inputs could systematically under-serve users whose language patterns are well-represented in training data.

Uncertainty communication. Overconfident precision estimates could create a false sense of reliability. Rigorous calibration is critical.

7.5 On the Probability of Success

Table 4: Subjective probability estimates for key claims.

Claim	Estimated Probability
Core principles broadly correct for general intelligence	60–70%
This specific architecture is the right instantiation	5–10%
Outperforms Transformers on standard benchmarks within 5 years	10–15%
Ideas partially absorbed into Transformer-based systems	40–50%
Pure prediction error minimization is sufficient	20–30%
Training stability solvable with known techniques	50–60%

8 Conclusion

We have presented ARIA (Adaptive Recurrent Integrative Architecture), a neural architecture based on the principle that perception, memory, and reasoning are inseparable aspects of a single process: hierarchical prediction error minimization over multiple timescales with memory consolidation. ARIA's key contributions are:

(1) A **Predictive State Column** that unifies bottom-up processing, top-down prediction, and memory access with learned precision weighting.

(2) A **multi-timescale hierarchy** that architecturally imposes temporal abstraction across four levels.

(3) A **tripartite memory substrate** with episodic storage, working memory for active reasoning, and semantic memory in the network weights, connected by a biologically-inspired consolidation cycle.

(4) **Adaptive computation** emerging naturally from prediction error dynamics, without a separate halting mechanism.

(5) A **principled training objective** derived from variational free energy minimization.

We have been forthright about substantial challenges: training stability, credit assignment, memory management at scale, the sufficiency of prediction error as a sole objective, unknown scaling properties, and engineering complexity. Nevertheless, the underlying principle—that cognition is prediction error minimization at multiple timescales—is worth pursuing as an architectural north star. The question is not whether prediction, memory, and reasoning should be unified; the brain demonstrates that they can be. The question is whether we can build artificial systems that realize this unification efficiently and at scale. ARIA is our attempt to sketch what such a system might look like.

References

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, et al. Flamingo: A visual language model for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to ponder. *ICML Workshop on Theoretic Foundation, Criticism, and Application Trend of Explainable AI*, 2021.

Andre M. Bastos, W. Martin Usrey, Rick A. Adams, George R. Mangun, Pascal Fries, and Karl J. Friston. Canonical microcircuits for predictive coding. *Neuron*, 76(4):695–711, 2012.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2022.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33: 1877–1901, 2020.

Christopher L. Buckley, Chang Sub Kim, Simon McGregor, and Anil K. Seth. The free energy principle for action and perception: A mathematical review. *Journal of Mathematical Psychology*, 81:55–79, 2017.

Rishidev Chaudhuri, Kenneth Knoblauch, Marie-Alice Gariel, Henry Kennedy, and Xiao-Jing Wang. A large-scale circuit mechanism for hierarchical dynamical processing in the primate cortex. *Neuron*, 88(2):419–431, 2015.

Junyoung Chung, Sungjin Ahn, and Yoshua Bengio. Hierarchical multiscale recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Andy Clark. Whatever next? predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Sciences*, 36(3):181–204, 2013.

Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Harriet Feldman and Karl Friston. Attention, uncertainty, and free-energy. *Frontiers in Human Neuroscience*, 4:215, 2010.

Karl Friston. A theory of cortical responses. *Philosophical Transactions of the Royal Society B*, 360(1456):815–836, 2005.

Karl Friston. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

Karl Friston, Thomas FitzGerald, Francesco Rigoli, Philipp Schwartenbeck, and Giovanni Pezzulo. Active inference and learning. *Neuroscience & Biobehavioral Reviews*, 68:862–879, 2016.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.

Uri Hasson, Janice Chen, and Christopher J. Honey. Hierarchical process memory: Memory as an integral component of information processing. *Trends in Cognitive Sciences*, 19(6):304–313, 2015.

Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *International Conference on Machine Learning (ICML)*, 2017.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

Georg B. Keller and Thomas D. Mrsic-Flogel. Predictive processing: A canonical cortical computation. *Neuron*, 100(2):424–435, 2018.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Jan Koutník, Klaus Greff, Faustino Gomez, and Jürgen Schmidhuber. A clockwork RNN. In *International Conference on Machine Learning (ICML)*, 2014.

Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.

Gina R. Kuperberg and T. Florian Jaeger. What do we mean by prediction in language comprehension? *Language, Cognition and Neuroscience*, 31(1):32–59, 2016.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

William Lotter, Gabriel Kreiman, and David Cox. Deep predictive coding networks for video prediction and unsupervised learning. In *International Conference on Learning Representations (ICLR)*, 2017.

James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

George A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.

Beren Millidge, Anil Seth, and Christopher L. Buckley. Predictive coding: A theoretical and experimental review. *arXiv preprint arXiv:2107.12979*, 2022.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.

John D. Murray, Alberto Bernacchia, David J. Freedman, Ranulfo Romo, Jonathan D. Wallis, Xinying Cai, et al. A hierarchy of intrinsic timescales across primate cortex. *Nature Neuroscience*, 17(12):1661–1663, 2014.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, et al. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length generalization. In *International Conference on Learning Representations (ICLR)*, 2022.

Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, et al. Hopfield networks is all you need. 2021.

Rajesh P. N. Rao and Dana H. Ballard. Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.

Tommaso Salvatori, Yuhang Song, Yanbo Hong, Lei Sha, Simon Frieder, Zhenghua Xu, Rafal Bogacz, and Thomas Lukasiewicz. Brain-inspired machine intelligence: A survey of neurobiologically-plausible credit assignment. *arXiv preprint arXiv:2312.09257*, 2024.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. The right tool for the job: Matching model and instance complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Hava T. Siegelmann and Eduardo D. Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50(1):132–150, 1995.

Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. BranchyNet: Fast inference via early exiting from deep neural networks. In *International Conference on Pattern Recognition (ICPR)*, 2016.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Haiguang Wen, Kuan Han, Junxing Shi, Yizhen Zhang, Eugenio Culurciello, and Zhongming Liu. Deep predictive coding network for object recognition. In *International Conference on Machine Learning (ICML)*, 2018.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations (ICLR)*, 2015.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning (ICML)*, 2017.

A Detailed Pseudocode

A.1 Full System Forward Pass

```
class ARIA:
    def __init__(self, config):
        self.levels = [PredictiveStateColumn(
            dim=config.dims[l],
            clock_rate=config.taus[l],
            transition=GatedRecurrentBlock(config, l),
            predictor_down=PredictionMLP(config, l),
            memory_query=QueryProjection(config, l),
            precision_gate=PrecisionEstimator(config, l)
        ) for l in range(config.L)]
        self.episodic_memory = EpisodicMemory(
            capacity=config.N_ep,
            key_dim=config.d_k,
            value_dim=config.d_v)
        self.working_memory = WorkingMemory(
            num_slots=config.K, slot_dim=config.d_w)
        self.output_head = OutputHead(config)
        self.convergence_estimator = ConvergenceEstimator(config)

    def forward(self, input_sequence, max_iter=16):
        outputs = []
        for t, x_t in enumerate(input_sequence):
            for iteration in range(max_iter):
                prev_states = [l.state.clone() for l in self.levels]
                # Bottom-up: compute prediction errors
                errors = []
                for l in range(len(self.levels)):
                    if not self.levels[l].should_tick(t):
                        errors.append(None); continue
                    pred_down = self.levels[l].predictor_down(
                        self.levels[l].state)
                    error = (x_t if l == 0 else
                             self.levels[l-1].state) - pred_down
                    errors.append(error)
                # Top-down: generate predictions
                predictions = []
                for l in range(len(self.levels)):
                    pred = (self.levels[l+1].predictor_down(
                                self.levels[l+1].state)
                            if l < len(self.levels)-1
                            else torch.zeros_like(self.levels[l].state))
                    predictions.append(pred)
                # Update all levels
                for l in range(len(self.levels)):
                    if not self.levels[l].should_tick(t): continue
                    query = self.levels[l].memory_query(
                        self.levels[l].state)
                    mem_content = self.episodic_memory.read(query)
                    pi = self.levels[l].precision_gate(
                        self.levels[l].state)
                    self.levels[l].state = (
                        self.levels[l].state
                        + self.levels[l].alpha
                        * self.levels[l].transition(
                            self.levels[l].state,
                            pi.bottom_up * errors[l],
                            pi.top_down * predictions[l],
                            pi.memory * mem_content,
                            self.working_memory.read_all()))
```

```
                # Working memory writes
                for l in range(self.config.l_min_write, len(self.levels)):
                    if self.levels[l].should_tick(t):
                        wc = self.levels[l].write_head(
                            self.levels[l].state)
                        self.working_memory.write(wc)
                # Convergence check
                delta = sum(torch.norm(self.levels[l].state
                                       - prev_states[l])
                            for l in range(len(self.levels)))
                conf = self.convergence_estimator(
                    [l.state for l in self.levels],
                    self.working_memory.read_all())
                if (delta < self.config.delta_converge and
                        conf > self.config.delta_confidence):
                    break
            # Episodic memory write
            total_err = sum(torch.norm(e)
                            for e in errors if e is not None)
            importance = torch.sigmoid(
                self.importance_net(total_err))
            self.episodic_memory.write(
                key=self.key_encoder(
                    [l.state for l in self.levels]),
                value=self.value_encoder(
                    [l.state for l in self.levels],
                    self.working_memory.read_all()),
                importance=importance)
            outputs.append(self.output_head(
                [l.state for l in self.levels],
                self.working_memory.read_all()))
        return outputs
```

Listing 1: ARIA full forward pass.

A.2 Consolidation Loop

```
def consolidate(self, n_episodes=32, eta_consolidate=1e-5):
    """Run one consolidation cycle."""
    episodes = self.episodic_memory.prioritized_sample(n_episodes)
    total_loss = 0
    for episode in episodes:
        with torch.enable_grad():
            replayed_output = self.forward(
                episode.context,
                max_iter=episode.original_iterations)
            replay_loss = F.cross_entropy(
                replayed_output, episode.targets)
            ewc_loss = sum(
                (self.fisher_information[n]
                 * (p - self.reference_params[n]) ** 2).sum()
                for n, p in self.named_parameters()
                if n in self.fisher_information)
            loss = replay_loss + self.config.lambda_ewc * ewc_loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(
                self.parameters(), max_norm=0.5)
            for param in self.parameters():
                if param.grad is not None:
                    param.data -= eta_consolidate * param.grad
                    param.grad.zero_()
        total_loss += loss.item()
```

```
    for episode in episodes:
        episode.importance *= self.config.consolidation_decay
    return total_loss / n_episodes
```

Listing 2: Consolidation loop.

B Hyperparameter Sensitivity Analysis (Proposed)

Table 5: Proposed hyperparameter sensitivity study for Phase 1.

Hyperparameter	**Range to Test**	**Expected Sensitivity**
Number of levels L	$\{2, 3, 4, 5, 6\}$	Moderate
Clock ratio r	$\{2, 4, 8\}$	High
State dimension ratio d_{l+1}/d_l	$\{0.5, 1.0, 1.5, 2.0\}$	Moderate
Working memory slots K	$\{3, 5, 7, 9, 12\}$	High (reasoning tasks)
Episodic memory capacity N_{ep}	$\{1\text{K}, 8\text{K}, 64\text{K}, 512\text{K}\}$	Moderate
Max iterations	$\{2, 4, 8, 16, 32\}$	High
Precision initialization	$\{0.1, 0.5, 1.0, 2.0\}$	High
Level loss weights λ_l (geometric decay)	$\{0.5, 0.7, 0.9\}$	Moderate
Convergence threshold δ	$\{0.001, 0.01, 0.1\}$	Moderate
Consolidation frequency	$\{100, 1000, 10000 \text{ steps}\}$	High (continual learning)

C Theoretical Connections

C.1 Relationship to the Free Energy Principle

ARIA is formally situated within the Free Energy Principle [Friston, 2010]:

$$F = \underbrace{D_{\text{KL}}[q(s) \| p(s \mid x)]}_{\geq 0} - \underbrace{\log p(x)}_{\text{log-evidence}} . \tag{38}$$

Therefore $F \geq -\log p(x)$. Minimizing F w.r.t. $q(s)$ makes q a better approximation to the true posterior (perception), and minimizing F w.r.t. the generative model parameters makes the model better at predicting observations (learning). Under the Laplace approximation and a hierarchical generative model, free energy minimization yields exactly the predictive coding dynamics implemented by ARIA.

C.2 Relationship to Recurrent State Spaces

ARIA's within-level dynamics can be expressed in state-space form:

$$s_l(t+1) = A_l(s_l(t))\, s_l(t) + B_l(s_l(t))\, u_l(t), \tag{39}$$

where A_l and B_l are state-dependent matrices and $u_l(t)$ collects precision-weighted inputs. This is analogous to the selective state space mechanism in Mamba [Gu and Dao, 2023], with the key extension that $u_l(t)$ includes top-down predictions, prediction errors, and memory contents.